

The Second Brain Security Contract

Second Brain is a microkernel with a boundary. On the other side live plugins, which consist of arbitrary code. If Second Brain is like the brain and plugins are cybernetic augmentations, then the immune system is like the security system. It mediates the boundary between the two and raises the alarm when attention is needed.

Two ways to run (arbitrary) code:

1. In-process
 - a. Unlimited CPU, RAM, and file access
2. In a Subprocess
 - a. Limits CPU, RAM, and what's visible on the DISK's files
 - b. Takes time to set up, uses additional resources

The code functions the same either way, just with different levels of security.

The code may interact with the outside environment using typed Requests. Before code runs, it is validated using a script. It automatically rejects any attempt to interact with the outside environment except via Requests. For example, `file.read()` would be rejected. However, a file could be read by yielding to the kernel with a `Read(file)` Request. Therefore, each request represents a capability of the kernel.

A complication arises when code imports either a library or another file. The other file can be validated using the same script as the original. However, libraries cannot be reliably validated since they are often in other languages or in binary. Importing those libraries is a security complication because they can access the CPU, RAM, and DISK without using Requests. This risk can be minimized by subprocessing the code with the foreign library. However, there is no way to transform a library's actions into requests.¹

When code is subprocessed or run in-process using the validation script, it is called sandboxing. Sandboxing means restricting the capabilities of the code to protect the main system (the kernel). To make this easier, the code is given an SDK (software development kit). It exposes the Requests system to the isolated code and also provides a list of common methods which may be helpful. For example: truncating characters, computing cosine similarity scores, or making markdown tables.

Sandboxed code can be run as a persistent background process, or as a temporary computation. In the first case, the sandbox container is left open indefinitely. In the second, the container is removed after use. Persistent containers can hold an internal state for periods of time, which may be useful.

APPENDIX A - A List of Second Brain's Capabilities Which May Be Turned Into Requests:

1. Read file
2. Write to file
3. Read SQL database
4. Write to SQL database
5. Run subagent

¹ The code can still be run, just with a disclaimer.

6. Register plugin
7. Deregister plugin
8. Call a Service's method
9. Load service
10. Unload service
11. Read config
12. Write to config
13. List directories
14. Read conversation context
15. Ask user question
16. Schedule cron job
17. Delete scheduled cron job
18. Edit scheduled cron job
19. Terminate/Respond (yes, the sandboxed code has to ask to end its own life). Invalid for persistent ones.
20. Reload plugin
21. HTTP request
22. Bash/terminal
23. Call slash command
24. Creating a conversation
25. Deleting a conversation
26. Iterating a conversation with enact()
27. Using conversation runtime hooks
28. (and others)

Note: this list can be simplified and re-categorized. For instance, 'read file', 'read SQL database', and 'read conversation' could be lumped under a single Read Request.

The security level of a Request depends on the nature of the Request, who's asking, and where it's headed. Using this information, the kernel policy function computes the security level of a given Request. There are two security levels: safe and unsafe. Unsafe Requests will automatically trigger a message to the user asking them to approve the request. Safe Requests will execute automatically. Thus, the main security boundary is asking the user for approval. When a request handles private information such as secrets and passwords, it receives another level of security.

The main reason a security system is needed is because Second Brain is an agentic system which can extend itself by writing the plugin code which hooks into the central kernel. It's a self evolution system, and without the careful requests system, then the agent could decide to extend itself without bounds.

Agent LibOS, a recent research paper, says it best:

*A self-evolving agent may change what it can ask for;
but it cannot thereby change what it is authorized to
affect.*

Since agents almost always have good intentions due to their RLHF training, the main concern is stupidity, not maliciousness. This is why it's important to make plugins easy to write and create, not hard.

It's also the reason that absolute security isn't necessary — the validation script technically can't be perfect and that's OK. If security is the main concern, then using a subprocess is the best option.

What requests return to the code: information. However, they can only return this information in the form of simple data types, not complex Python objects. If the return data is in the form of a complex object such as a dataclass, it must be turned into a dictionary before crossing the boundary. When a Request is for taking an action such as writing to a file, the returned information is a report on whether that action succeeded or failed.

Plugins can make Requests for other plugins. The chain of requests from one plugin to another is called the 'Chain of Provenance'. When plugin A calls on plugin B, the chain is written down. It also simplifies and keeps track of permission dialogs for the user, since the user is able to see where each Request originated.